Basic matrix multiplication in Python Java and C

Assignment 1 - Universidad de Las Palmas de Gran Canaria

Student: Ivan Giacomo Colella

Abstract

This study compares execution time and process memory for basic dense matrix multiplication in Python, Java and C on one Windows computer. All implementations use explicit i–j–k loops, flat row-major storage and 64-bit floating-point values. The final experiment covers matrix dimensions 64, 128, 256 and 512, with three independent processes per language and dimension. Each process requests five measured samples after warm-up. Of 36 processes, 33 completed; all three Python runs at n = 512 exceeded the 120 s process budget. At n = 256, the median of process medians was 2411.224 ms for Python, 27.624 ms for Java and 24.818 ms for C. C had the lowest aggregate kernel time and process memory peak among completed configurations. These results apply to the tested implementations and environment; they do not establish a universal ranking of programming languages.

1 Introduction and background

The research question is: how does matrix dimension affect kernel runtime and peak process resident memory for comparable basic implementations in Python, Java and C? The independent factors are language implementation and matrix dimension. The outcomes are elapsed kernel time, process memory peak and completion within fixed resource budgets. This follows the empirical scope of Assignment 1[1].

Each output element is a dot product of a row of A and a column of B. The three nested loops perform n^3 multiply-and-accumulate iterations, giving $\Theta(n^3)$ arithmetic work. Three dense arrays contain $24n^2$ bytes of numeric elements, giving $\Theta(n^2)$ array storage. The initial prediction is approximately cubic runtime growth, with different constants between implementations. Resident process memory need not follow the array-storage estimate because it also includes the runtime and temporary allocations.

2 Experimental environment

Component	Recorded configuration
CPU	Intel Core i7-10510U at nominal 1.80 GHz; 4 cores, 8 logical processors
Memory and OS	15.83 GiB RAM; Windows 11 Home 10.0.26300, 64-bit
Language tools	Python 3.13.5; Temurin JDK 21.0.12.1; GCC 16.2.0, UCRT64
Python packages	NumPy 2.5.4; psutil 7.2.2; Matplotlib 3.10.8
Execution	PyCharm terminal; one application computation thread

3 Methodology

3.1 Comparable implementations and inputs

Python uses array('d'), Java uses double[] and C uses allocated double arrays. Each stores element (i, j) at offset i*n + j. All kernels use the same i–j–k order, reset a scalar accumulator for each output element and overwrite C. A and B and the reusable output array are allocated before measurement. Python still performs indexing and arithmetic through its runtime; identical element storage does not imply identical execution overhead. No matrix multiplication library replaces the required loops.
Shared text inputs contain n followed by A and B. A Python random.Random generator uses seed 42 + n and draws integers uniformly from −5 to 5, converted to floating point, filling A before B. Files were generated for n = 16, 32, 64, 128, 256 and 512. The final performance sequence uses 64, 128, 256 and 512; the smaller sizes support validation. SHA-256 checks confirm that all methods use the same input file for each size.

3.2 Correctness checks

All three implementations passed six elementwise tests: a known 2 × 2 product, multiplication by the identity, multiplication by zero, a fractional example, and random cases of sizes 16 and 32 checked against NumPy matrix multiplication. A value is accepted when its absolute error is at most the larger of 10^{-12} and 10^{-12} times the larger magnitude of the actual and reference values. These tight tolerances suit the small bounded test data; the fractional case uses exactly representable binary fractions. They are not a general error bound for arbitrary input distributions. Benchmark checksums also agree across methods, but checksums do not replace elementwise validation.

3.3 Timing and repetitions

Timers are time.perf_counter_ns in Python[2], System.nanoTime in Java[3] and QueryPerformanceCounter with QueryPerformanceFrequency in C[4]. Each timed region contains one call to the multiplication kernel, including scalar initialization and output writes. File loading, array allocation, checksum calculation and result output are outside that region. Timer and function-call overhead remain included. Results are consumed by checksum calculation after measured calls.

Python and C use 3 warm-up calls; Java uses 100. Pilot Java measurements motivated the longer warm-up, but a fixed count does not prove steady-state compilation. Each process requests five samples. At n = 64, a sample averages eight individually timed calls; at larger sizes it contains one call. This is not a single timer around the whole batch.

Three fresh processes are run for each language–size pair. Within each size, the execution order rotates through Python–Java–C, Java–C–Python and C–Python–Java. Final results are kept separate from pilot and validation runs. Within each process, the median and interquartile range (IQR) summarize the five samples. The main estimate is the median of the three process medians; variability bars show their minimum and maximum. The 15 samples are not treated as 15 independent processes.

3.4 Resource limits and memory boundary

The monitor imposes a 120 s elapsed-process budget and a 1024 MiB resident-memory budget using polling. A worker emits READY after measurement and waits for RELEASE, allowing the parent to read the cumulative Windows peak working-set counter via psutil before exit. The reported peak covers startup through READY, including input loading, runtime, warm-ups and output. It excludes the parent monitor and shutdown after RELEASE. Limits are soft because enforcement occurs at polling points. Partial samples are flushed to disk and retained if a process times out.

4 Runtime results

Table 1 summarizes the final experimental records[5] and reports the median of the three process medians, followed by their minimum–maximum range in brackets. All complete configurations have three processes and five samples per process. Values are milliseconds per multiplication. A timeout has no substituted runtime estimate.

Table 1. Kernel time in ms with the range of process medians.

n	Python	Java	C
64	50.200 [41.392–62.226]	0.403 [0.361–0.473]	0.275 [0.253–0.344]
128	306.723 [298.639–314.290]	2.176 [2.090–2.352]	1.929 [1.817–2.216]
256	2411.224 [2238.821–2486.857]	27.624 [25.400–28.402]	24.818 [24.111–29.285]
512	Timeout 3 of 3 processes	315.729 [307.434–383.322]	220.900 [207.774–234.865]

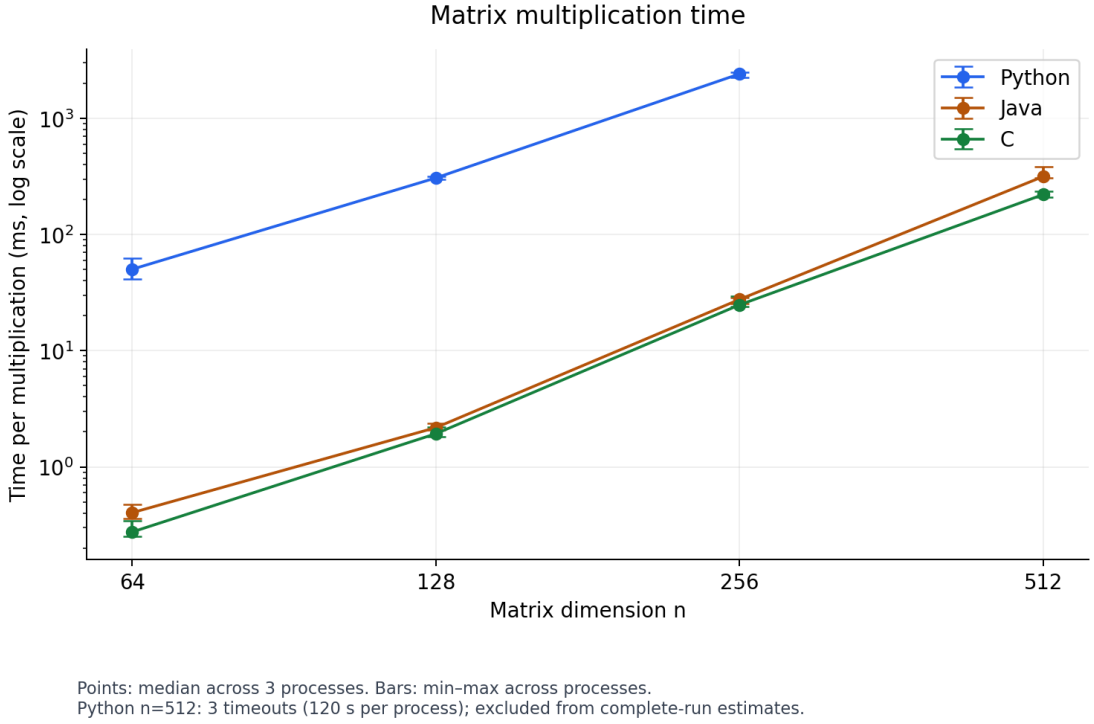


Figure 1. Kernel runtime across matrix dimensions. Both axes are logarithmic. Bars describe between-process variation, not confidence intervals.

At n = 256, Python requires approximately 97.16 times the C kernel time, while Java requires 1.11 times the C time. At n = 512, the Java estimate is 315.729 ms versus 220.900 ms for C, a ratio of 1.43. The difference between C and Java therefore changes with size. These are descriptive ratios of aggregate estimates; no statistical significance test was performed.

5 Scaling relative to cubic work

A doubling of n multiplies the loop iteration count by eight. Table 2 compares adjacent aggregate runtimes. Figure 2 divides each runtime by n^3; a constant ratio would be compatible with an exactly cubic timing curve over these sizes.

Table 2. Runtime ratio when matrix dimension doubles.

Size transition	Python	Java	C	Cubic prediction
64 to 128	6.11	5.40	7.01	8.00
128 to 256	7.86	12.69	12.87	8.00
256 to 512	Unavailable	11.43	8.90	8.00

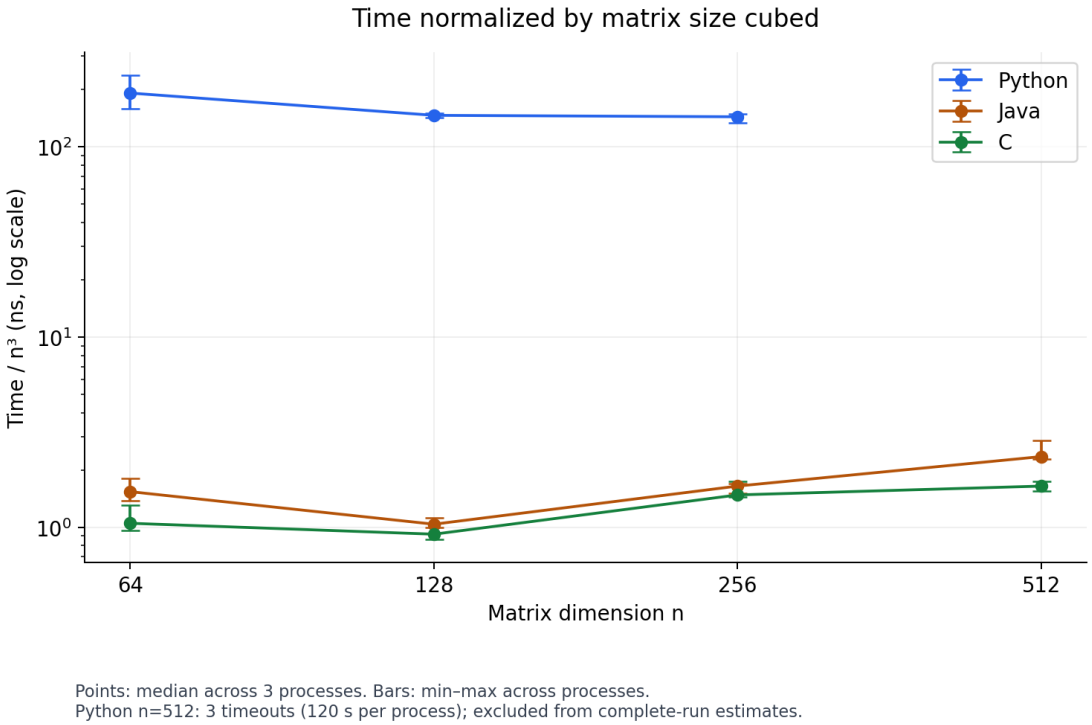


Figure 2. Kernel time divided by n^3. The normalized time is expressed in nanoseconds; both axes use logarithmic scaling.

Python grows by a factor of 7.86 between n = 128 and 256, close to the factor of eight predicted by the loop count. C grows by 12.87 over the same transition and by 8.90 from 256 to 512. Java grows by 12.69 and 11.43 for those transitions. Finite-size timing therefore does not follow one constant times n^3 exactly, although the implemented arithmetic work is cubic.

The fixed row-major i–j–k traversal accesses successive elements of A but strides through a column of B. This provides a plausible memory-access contribution as matrices grow. Runtime compilation, scheduling and processor frequency may also affect observations. The experiment did not collect cache counters, compilation traces or temperature data, so it cannot identify which mechanism caused a particular change.

6 Memory results and feasibility

Table 3 summarizes peak worker resident memory through READY. Entries are the median of three process peaks followed by their range. MiB denotes 2^{20} bytes. These values describe the full process rather than only the matrix arrays.

Table 3. Peak process resident memory in MiB.

n	Python	Java	C
64	13.812 [13.812–13.816]	77.809 [75.438–78.891]	4.254 [4.250–4.254]
128	15.129 [15.129–15.129]	109.770 [109.234–110.633]	4.535 [4.535–4.535]
256	20.273 [20.203–20.297]	206.219 [205.246–208.355]	5.668 [5.668–5.668]
512	Incomplete	325.969 [325.398–328.297]	10.168 [10.168–10.168]

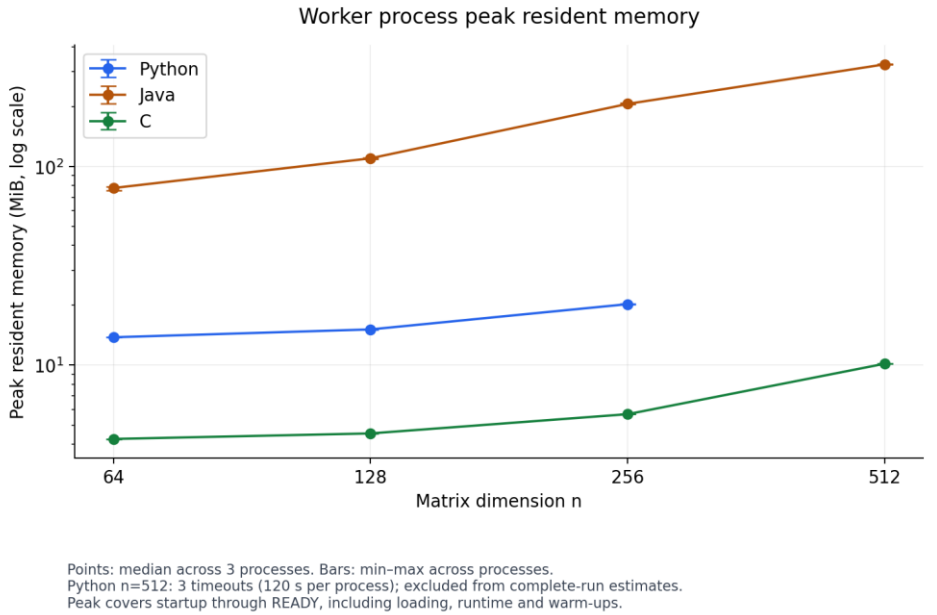


Figure 3. Cumulative worker memory peak measured at READY. Timeout observations aren't mixed with final-process peaks.

At n = 512, the three numeric arrays alone require 6 MiB. The measured C process median is 10.168 MiB and the Java median is 325.969 MiB. Runtime structures, parsing and temporary allocations contribute to this difference; the measurement cannot isolate their individual contributions. Java also performs more warm-ups. No final run was classified as a memory-budget failure.

The largest attempted size was 512 for every method. The largest size completing the full protocol in all three processes was 256 for Python and 512 for Java and C. Python at 512 saved 2, 1 and 0 of the requested five samples before timeout. Its three available samples range from 21.696 to 28.178 s per multiplication, but are incomplete observations and are excluded from the main aggregate comparisons. The 120 s timeout is not the duration of one multiplication.

7 Discussion and limitations

The experiment answers the research question within a shared environment: increasing matrix size raises kernel time and process memory, but the three implementations differ substantially in constant costs and practical completion limits. C has the lowest aggregate kernel time at each completed comparison. Java remains much closer to C than to Python, while its measured resident-memory peak is substantially higher.

The code supports a distinction between interpreted Python operations and compiled C and JVM execution, but the measured gap is not a controlled decomposition of interpreter overhead. Indexing, function calls, numeric operations and runtime behavior differ together. The implementations share an algorithm and numeric layout, not an identical machine instruction stream. The C build uses -O2 and -ffp-contract=off; no manual loop reordering, blocking or library multiplication is introduced. Compiler transformations under -O2 were not inspected at the assembly level.

There are three main sources of uncertainty. First, the experiment uses one laptop and three independent processes per configuration. CPU affinity, power policy, thermal state and background load were not recorded as controlled variables. The min–max bars describe the observed spread only. For example, Python at n = 64 has process medians from 41.392 to 62.226 ms, and Java at n = 512 ranges from 307.434 to 383.322 ms. These differences should remain visible rather than being removed as inconvenient observations.

Second, JVM compilation and garbage collection may affect execution and memory. The 100 warm-up calls are a declared protocol choice, not evidence that every size reached a stable state. Computation uses one application thread, but runtime service threads may still exist. No custom JVM heap or garbage-collector flags were supplied. NumPy is used for correctness references, not timed kernels; numerical-library threading therefore does not define the benchmark computation.

Third, the memory metric includes setup and warm-up, whereas timing covers only the kernel. Their scopes intentionally differ and must be interpreted separately. Repeated individually timed calls reduce reliance on one short observation, but do not eliminate timer overhead. Fixed process budgets also mean that slow implementations may spend much of the budget warming up; timeout results characterize feasibility of the complete declared protocol.

8 Conclusions

For the tested shared inputs and settings, C provides the lowest aggregate kernel time and resident-memory peak. Java has similar kernel performance at intermediate sizes but a larger process footprint. Python completes the full protocol through n = 256 and times out at n = 512; this is an experimental resource limit, not an absolute capability limit. The arithmetic remains cubic in all three implementations, while observed scaling also reflects finite-size and runtime effects. Further work could isolate loop order, cache behavior and JVM stabilization in separate experiments without replacing this baseline.

AI assistance disclosure

OpenAI ChatGPT supported experimental planning, code development and debugging. The measurements were produced by running the programs locally. AI-generated text was not used as experimental evidence.

9 Reproducibility materials

Repository: https://github.com/ivanc25/Assignment_01_BD.git. The accompanying project contains source code under src, shared inputs and metadata under data, measurement and analysis scripts under scripts, raw process records under results/final, derived CSV tables under results/analysis and figures under plots. Pilot and validation outputs are separate and are not pooled with final measurements.

From the project root in Windows PowerShell, use the recorded language toolchain and run the following commands. The final-loop command reproduces the full campaign and may take several minutes. Preserve the original raw data: new timestamped runs should be collected in a separate project copy because the analysis script expects exactly 36 final runs.

```
python -m pip install numpy==2.5.4 psutil==7.2.2 matplotlib==3.10.8
python scripts/generate_inputs.py
python scripts/prepare_reference_tests.py
javac src/java/MatrixJava.java
gcc -std=c11 -O2 -ffp-contract=off -Wall -Wextra src/c/matrix_c.c -o src/c/matrix_c.exe -lm
python src/python/matrix_python.py --test
java -cp src/java MatrixJava --test data
.\src\c\matrix_c.exe --test data

$orders = @("python,java,c", "java,c,python", "c,python,java")
foreach ($n in @(64,128,256,512)) {
    foreach ($order in $orders) {
        foreach ($method in $order.Split(",")) {
            python scripts/run_controlled.py $method --size $n --phase final
        }
    }
}
python scripts/analyze_results.py
```

To regenerate only tables and figures from the supplied measurements, run the last command alone. Each raw process directory retains summary.json, timings.csv, memory_samples.csv and stdout/stderr logs. runs.csv preserves per-process median and IQR, configurations.csv holds aggregate estimates, and separate timeout and partial-sample files retain interrupted observations. The final audit reports 36 processes, 33 completions, 3 timeouts and 168 saved samples: 165 from completed processes plus 3 partial samples. Shared input hashes, row counts, timing arithmetic, summary statistics and checksum agreement were verified.

References

[1] Universidad de Las Palmas de Gran Canaria, “Individual Assignments — Matrix Multiplication,” BIG DATA, academic year 2026–2027, revised proposal, pp. 2–4, 8–9. Course handout: BD_Individual_Assignments_1_2026_2027.pdf.

[2] Python Software Foundation, “time — Time access and conversions,” Python 3.13 documentation, sections perf_counter and perf_counter_ns. https://docs.python.org/3.13/library/time.html (accessed 13 October 2026).

[3] Oracle, “System,” Java Platform Standard Edition 21 API specification, method nanoTime. https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/lang/System.html (accessed 13 October 2026).

[4] Microsoft, “QueryPerformanceCounter function,” Windows API documentation. https://learn.microsoft.com/en-us/windows/win32/api/profileapi/nf-profileapi-queryperformancecounter (accessed 13 October 2026).

[5] Project experimental records, results/final and results/analysis, 13 October 2026. All numerical tables and figures in this report are derived from these records using scripts/analyze_results.py.